DAY 4

Advanced Python Programming Concepts

Easy-to-understand study notes, examples, practical tasks, and a complete Student Management Application

Topics covered

- Lists and tuples
- Sets and dictionaries
- Strings
- Functions and modules
- Exception handling
- File handling
- List and dictionary comprehensions
- Lambda functions and basic functional programming
- Student record processing project

Beginner-friendly English notes

Learning Objectives

By the end of Day 4, you should be able to store multiple records, select and process data, search and sort records, handle invalid input safely, and save information in a file.

1. Lists

A **list** stores multiple values in one variable. Lists are ordered, changeable (mutable), and allow duplicate values.

```
marks = [78, 85, 92, 85]
print(marks[0])       # 78
print(marks[-1])      # 85
marks.append(95)      # add an item
marks.remove(85)      # remove the first matching item
marks[0] = 80         # update an item
print(len(marks))     # number of items
```

Useful list operations

```
numbers = [5, 2, 9, 1]
numbers.append(7)
numbers.insert(1, 10)
numbers.sort()             # ascending order
numbers.sort(reverse=True) # descending order
numbers.reverse()
print(numbers)
print(9 in numbers)        # membership test
```

When to use: Use a list when you need an ordered collection that may change, such as a list of student names or marks.

2. Tuples

A **tuple** is an ordered collection that cannot be changed after creation (immutable). It can contain duplicate values.

```
coordinates = (18.5, 73.8)
student = ("S101", "Asha", 88)
print(student[1])     # Asha
print(len(student))   # 3

# A one-item tuple needs a comma:
one_item = (10,)
```

Use tuples for grouped values that should remain fixed. Lists are better when you need to add, remove, or update items.

3. Sets

A **set** stores unique values. It does not keep a reliable positional order, so do not access set items by index.

```
subjects = {"Python", "SQL", "Python"}
print(subjects)          # duplicate Python is stored once
subjects.add("HTML")
subjects.discard("SQL")  # safe if item is absent

a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # union: {1, 2, 3, 4, 5}
print(a & b)  # intersection: {3}
print(a - b)  # difference: {1, 2}
```

When to use: Use a set to remove duplicates or check unique values, such as unique course names.

4. Dictionaries

A **dictionary** stores information as key-value pairs. Keys are unique. It is useful for a record with named fields.

```
student = {
    "id": "S101",
```

```
    "name": "Asha",
    "marks": 88
}
print(student["name"])       # Asha
print(student.get("age", 0)) # 0 if age is missing
student["marks"] = 91        # update
student["city"] = "Pune"     # add
student.pop("city", None)    # remove safely
```

Loop through a dictionary

```
for key, value in student.items():
    print(key, ":", value)
```

A list of dictionaries is a good structure for multiple records:

```
students = [
    {"id": "S101", "name": "Asha", "marks": 88},
    {"id": "S102", "name": "Ravi", "marks": 76}
]
```

5. Strings

A **string** is text written inside quotes. Strings are immutable, meaning you create a new string rather than changing individual characters.

```
name = "  shweta mulage  "
print(name.strip())            # remove outer spaces
print(name.strip().title())    # Shweta Mulage
print(name.upper())
print(name.lower())
print(name.replace("shweta", "Sneha"))
print("Python" in "Learn Python")
print("A,B,C".split(","))      # ['A', 'B', 'C']
print("-".join(["A", "B", "C"])) # A-B-C

student_id = "S101"
print(f"Student ID: {student_id}")
```

String indexing and slicing

```
word = "PYTHON"
print(word[0])     # P
print(word[-1])    # N
print(word[1:4])   # YTH
```

6. Functions and Modules

A **function** is a reusable block of code. Define it using `def`. Parameters receive input; `return` sends a result back.

```
def calculate_percentage(marks, total=100):
    if total <= 0:
        raise ValueError("Total must be greater than zero")
    return (marks / total) * 100

percentage = calculate_percentage(85)
print(percentage)
```

A simple module

A module is a Python file containing reusable code. For example, save this in `helpers.py`:

```
# helpers.py
def greet(name):
    return f"Hello, {name}!"
```

Use it in another file in the same folder:

```
from helpers import greet
print(greet("Asha"))
```

Python also includes built-in modules. For example, `import json` supports JSON data and `import os` provides operating-system utilities.

7. Exception Handling

An exception is an error that occurs while a program is running. Use `try` and `except` to handle expected errors and keep the program from stopping unexpectedly.

```
try:
    marks = int(input("Enter marks (0-100): "))
    if not 0 <= marks <= 100:
        raise ValueError("Marks must be from 0 to 100")
    print("Marks:", marks)
except ValueError as error:
    print("Invalid input:", error)
finally:
    print("Input process finished.")
```

Use specific exception types such as `ValueError`, `FileNotFoundError`, and `json.JSONDecodeError`. Avoid catching every error without a reason.

8. File Handling

File handling lets a program store data permanently. The `with open(...)` pattern closes the file automatically.

Common modes: **r** = read, **w** = write/overwrite, **a** = append, **x** = create only if it does not exist.

```
# Write text (overwrites the file)
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Learning Python file handling.\n")

# Read text
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)

# Append text
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("This line is added later.\n")
```

Saving records as JSON

JSON is a convenient text format for lists and dictionaries. It is easier to read and write structured records than to build a custom text format.

```
import json

students = [{"id": "S101", "name": "Asha", "marks": 88}]
with open("students.json", "w", encoding="utf-8") as file:
    json.dump(students, file, indent=4)

with open("students.json", "r", encoding="utf-8") as file:
    students = json.load(file)
```

9. List and Dictionary Comprehensions

A comprehension creates a new collection in a compact, readable way.

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [n * n for n in numbers]
evens = [n for n in numbers if n % 2 == 0]
print(squares)  # [1, 4, 9, 16, 25, 36]
print(evens)    # [2, 4, 6]

names = ["asha", "ravi"]
capital_names = [name.title() for name in names]

marks = {"Asha": 88, "Ravi": 76, "Neha": 93}
passed = {name: score for name, score in marks.items()
          if score >= 40}
print(passed)
```

Keep comprehensions simple. Use a normal loop if the logic becomes difficult to understand.

10. Lambda Functions and Basic Functional Programming

A **lambda** is a small anonymous function written in one expression. It is often used as a short sorting key.

```
double = lambda number: number * 2
print(double(5))  # 10

students = [
    {"id": "S101", "name": "Ravi", "marks": 76},
    {"id": "S102", "name": "Asha", "marks": 88}
]
by_name = sorted(students, key=lambda s: s["name"])
by_marks = sorted(students, key=lambda s: s["marks"], reverse=True)
print(by_name)
print(by_marks)
```

Other functional tools include `map()` (transform values) and `filter()` (select values). List comprehensions are often clearer for beginners.

```
numbers = [1, 2, 3, 4]
doubled = list(map(lambda n: n * 2, numbers))
greater_than_two = list(filter(lambda n: n > 2, numbers))
print(doubled)            # [2, 4, 6, 8]
print(greater_than_two)   # [3, 4]
```

11. Practical Project: Student Management Application

This console application uses a list of dictionaries to store student records. It can add, display, search, sort, update, and delete records. It saves data in `students.json`, so records remain available when you run the program again.

How to run

1. Install Python 3.
2. Create a folder named Day4_Student_App.
3. Create a file named student_app.py in that folder.
4. Copy the complete code below into student_app.py.
5. Open a terminal in that folder and run:
   python student_app.py

Complete code

```
import json
from pathlib import Path

DATA_FILE = Path("students.json")


def load_students():
    """Load student records from JSON, or start with an empty list."""
    if not DATA_FILE.exists():
        return []
    try:
        with DATA_FILE.open("r", encoding="utf-8") as file:
            data = json.load(file)
        if not isinstance(data, list):
            print("Warning: data file format is invalid. Starting empty.")
            return []
        return data
    except (json.JSONDecodeError, OSError) as error:
        print(f"Could not read saved data: {error}")
        return []


def save_students(students):
    """Save all records to the JSON file."""
    try:
        with DATA_FILE.open("w", encoding="utf-8") as file:
            json.dump(students, file, indent=4)
    except OSError as error:
        print(f"Could not save data: {error}")


def find_by_id(students, student_id):
    for student in students:
        if student["id"].lower() == student_id.lower():
            return student
    return None


def add_student(students):
    student_id = input("Enter student ID: ").strip()
    if not student_id:
        print("ID cannot be empty.")
        return
    if find_by_id(students, student_id):
        print("A student with this ID already exists.")
        return

    name = input("Enter student name: ").strip()
    if not name:
        print("Name cannot be empty.")
        return

    try:
        marks = float(input("Enter marks (0-100): "))
        if not 0 <= marks <= 100:
            raise ValueError("Marks must be between 0 and 100.")
    except ValueError as error:
        print(f"Invalid marks: {error}")
        return

    students.append({"id": student_id, "name": name, "marks": marks})
```

```
    save_students(students)
    print("Student added successfully.")


def display_students(students):
    if not students:
        print("No student records found.")
        return
    print("\n{:<12} {:<22} {:>8}".format("ID", "NAME", "MARKS"))
    print("-" * 44)
    for student in students:
        print("{:<12} {:<22} {:>8.2f}".format(
            student["id"], student["name"], student["marks"]
        ))


def search_student(students):
    term = input("Enter student ID or name: ").strip().lower()
    matches = [
        student for student in students
        if student["id"].lower() == term
        or term in student["name"].lower()
    ]
    display_students(matches)


def sort_students(students):
    print("1. Sort by name")
    print("2. Sort by marks (highest first)")
    choice = input("Choose sorting option: ").strip()

    if choice == "1":
        ordered = sorted(students, key=lambda s: s["name"].lower())
    elif choice == "2":
        ordered = sorted(students, key=lambda s: s["marks"], reverse=True)
    else:
        print("Invalid sorting option.")
        return
    display_students(ordered)


def update_student(students):
    student_id = input("Enter ID of student to update: ").strip()
    student = find_by_id(students, student_id)
    if student is None:
        print("Student not found.")
        return

    new_name = input(f"New name [{student['name']}]: ").strip()
    if new_name:
        student["name"] = new_name

    new_marks = input(f"New marks [{student['marks']}]: ").strip()
    if new_marks:
        try:
            marks = float(new_marks)
            if not 0 <= marks <= 100:
                raise ValueError("Marks must be between 0 and 100.")
            student["marks"] = marks
        except ValueError as error:
            print(f"Invalid marks: {error}")
            return

    save_students(students)
    print("Student updated successfully.")


def delete_student(students):
    student_id = input("Enter ID of student to delete: ").strip()
    student = find_by_id(students, student_id)
    if student is None:
        print("Student not found.")
        return

    students.remove(student)
    save_students(students)
    print("Student deleted successfully.")


def main():
```

```
    students = load_students()

    while True:
        print("\n===== STUDENT MANAGEMENT SYSTEM =====")
        print("1. Add student")
        print("2. Display all students")
        print("3. Search by ID or name")
        print("4. Sort records")
        print("5. Update student")
        print("6. Delete student")
        print("7. Exit")

        choice = input("Enter your choice (1-7): ").strip()

        if choice == "1":
            add_student(students)
        elif choice == "2":
            display_students(students)
        elif choice == "3":
            search_student(students)
        elif choice == "4":
            sort_students(students)
        elif choice == "5":
            update_student(students)
        elif choice == "6":
            delete_student(students)
        elif choice == "7":
            save_students(students)
            print("Data saved. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number from 1 to 7.")


if __name__ == "__main__":
    main()
```

Sample run

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add student
2. Display all students
3. Search by ID or name
4. Sort records
5. Update student
6. Delete student
7. Exit
Enter your choice (1-7): 1
Enter student ID: S101
Enter student name: Asha
Enter marks (0-100): 88
Student added successfully.

ID           NAME                      MARKS
-------------------------------------------
S101         Asha                      88.00
```

12. How the Application Works

• **List:** The variable `students` holds all student records.

• **Dictionary:** Each student has keys such as `id`, `name`, and `marks`.

• **Functions:** Separate functions keep each operation organised and reusable.

• **Exception handling:** Invalid marks and file errors are handled with useful messages.

• **Comprehension:** Search uses a list comprehension to find matching records.

• **Lambda:** Sorting uses a lambda to choose the name or marks field.

• **File handling:** JSON saves and loads records between runs.

13. Practice Tasks

- Create an Employee Management Application with employee ID, name, department, and salary.
- Search for an employee by ID or part of their name.
- Sort employees by name and salary.
- Reject negative salaries and duplicate employee IDs.
- Save employee data to a JSON file and load it at startup.
- Use a set to display unique department names.
- Use a dictionary comprehension to create a name-to-salary mapping.

14. Quick Revision

Concept	Meaning	Example
List	Ordered and changeable	[10, 20, 30]
Tuple	Ordered and immutable	(10, 20, 30)
Set	Unique values	{10, 20, 30}
Dictionary	Key-value pairs	{"name": "Asha"}
Function	Reusable block of code	def greet(): ...
Exception handling	Manage runtime errors	try / except
File handling	Read or write stored data	open(), json.load()
Comprehension	Compact collection creation	[n*n for n in nums]
Lambda	Small anonymous function	lambda n: n * 2

Day 4 expected output: Collection-based Python programs and a working Student Management Application that supports adding, searching, sorting, updating, deleting, and saving student records.